

Readme

- **Nom:** HOUNLEBA Lenaïc Love (L3-IFA)
- **Date :** 02 décembre 2025
- **Intitulé:** Système de réservation de véhicules
- **Template Bootstrap front-office : Rentaly:**
<https://www.madebydesignesia.com/themes/rentaly/index.html>
- **Template Bootstrap back-office : WowDash :**
<https://wowdash.wowtheme7.com/demo/index-2.html>
- **Url de l'application web:** <https://obiwan.univ-brest.fr/~e22507733/>
- **Commandes pour la sauvegarde des fichiers de l'application Web:**

Sauvegarde sur GitLab

```
cd V2
git init .
git status
git add .
git status
git commit -m "V2 finalisée de l'application Web"
git config user.name "Nom Prénom"
git config user.email mon.mail@etudiant.univ-brest.fr
git branch V2
git checkout V2
git remote add gitlab https://gitlab-depinfo.univ-brest.fr/xxxxxxxxx/resaweb.git
git push gitlab V2
```

Mise à jour ultérieure d'un fichier

```
git status
git add ci/app/Config/Routes.php
git commit -m "Mise à jour fichier Routes"
```

git push gitlab V2

Sur Vador

ssh votrelogin@vador-fs-lic.univ-brest.fr

mkdir projet_isi

cd projet_isi

mkdir V1_sauv

mkdir V2_sauv

mkdir V3_sauv

Depuis Obiwan :

ssh exxxxxxxx@obiwan.univ-brest.fr

cd V2

scp -r ci/ votrelogin@vador-fs-lic.univ-brest.fr:projet_isi/V2_sauv

Sauvegarde locale sur Obiwan

cd ..

mkdir V2_copie

cp -r V2/ci V2_copie

ln -s V2_copie prod

➤ Comptes créés pour tester l'application:

– Membres:

- adrien.bernard : @@@Hello###M
- sophie.renaud : HelloSophieRenaud

– Administrateurs:

- sophie.moreau : JohnDoe123
- anne.martin : HelloAnneMartin
- principal: princ25*!ASER

– Invités:

- Invite3 : HelloCherInvite
- Invite5 : HelloMrinvite

➤ Court descriptif de la version en ligne (V2.1)

Durant ce projet, j'ai progressivement construit les sprints 1 et 2 de l'application ResaWeb, en commençant par les bases du Sprint 1. J'ai d'abord mis en place l'affichage des actualités sur la page d'accueil ainsi que la gestion des questions des visiteurs, avec la possibilité pour eux d'envoyer une demande via un formulaire. Ensuite, j'ai intégré la connexion administrateur : une fois connecté, l'administrateur peut consulter ses informations personnelles, visualiser ses réservations à venir, accéder aux messages des visiteurs et y répondre, et créer de nouveaux comptes « invités ». J'ai poursuivi en ajoutant la connexion membre, permettant à chaque utilisateur authentifié d'accéder à son espace privé, de consulter son profil, de voir ses réservations, et d'afficher la liste de tous les adhérents. À partir du Sprint 2, j'ai enrichi l'application avec la gestion des ressources : affichage des ressources disponibles, ajout d'une nouvelle ressource et suppression de celles existantes par les administrateurs. Enfin, j'ai développé la fonctionnalité permettant d'afficher les réservations d'une journée spécifique, aussi bien pour les administrateurs que pour les membres, en respectant les contraintes de sécurité, de validation et d'organisation demandées dans les user stories.

➤ **Nom de la base de données:** e22507733_db1

➤ **Court descriptif des triggers, fonction, procédure, vue créés:**

Dans le cadre du projet, j'ai également mis en place plusieurs éléments SQL avancés afin de renforcer la cohérence des données et d'automatiser certaines opérations liées aux réservations. J'ai d'abord créé une vue regroupant toutes les informations importantes d'une réservation, ce qui m'a permis de rassembler en une seule structure la ressource concernée, le conducteur et les passagers. Cette vue facilite beaucoup les contrôles et la compréhension des données, car elle centralise ce qui est habituellement réparti sur plusieurs tables.

J'ai aussi développé une fonction SQL permettant de savoir rapidement combien de réservations une ressource possède pour une date précise. Cette fonction est utilisée pour effectuer des contrôles métier, et elle illustre la manière dont il est possible d'interroger efficacement la base avec des paramètres.

Ensuite, j'ai réalisé une procédure stockée capable d'inscrire automatiquement un membre à une réservation, avec une gestion fine des cas particuliers. Cette procédure reçoit l'identifiant du membre et celui de la réservation, et elle renvoie un code indiquant si l'inscription est acceptée, refusée parce que la jauge de la ressource est déjà pleine, ou rejetée parce que la réservation n'existe pas. C'est une manière propre d'encapsuler une règle métier au niveau de la base de données.

Enfin, j'ai mis en place plusieurs triggers destinés à garantir la cohérence des données sur le long terme. Le premier empêche tout simplement qu'un même membre puisse être inscrit deux fois à la même réservation. J'ai également ajouté un trigger dédié à la mise à jour automatique du statut d'une réservation (Planifiée, En cours, Terminée, Annulée) dès qu'elle est créée ou

modifiée. Ce mécanisme permet d'éviter les incohérences, surtout lorsque des réservations vieillissent dans le temps.

Au final, ces éléments SQL permettent d'avoir une base de données plus intelligente, capable de gérer automatiquement un certain nombre de contraintes, tout en allégeant le code PHP et en sécurisant les opérations clés du projet.

➤ **Ce qu'il reste à faire et les améliorations à apporter :**

Même si j'ai déjà finalisé l'essentiel des fonctionnalités demandées dans les Sprints 1 et 2 (connexion complète, gestion des comptes, affichage des ressources, ajout/suppression de ressources, affichage des réservations par date pour les membres et les administrateurs, etc.), il reste encore plusieurs éléments importants à mettre en place pour compléter pleinement l'application ResaWeb.

La prochaine étape serait d'implémenter la gestion complète des réservations, notamment la possibilité pour un membre ou un administrateur de s'inscrire à un créneau, de se retirer d'un créneau, et de saisir un bilan pour une séance déjà passée. Aujourd'hui, ces interactions ne sont pas encore actives dans mon application, même si la structure de la base de données est entièrement prête à les accueillir.

Il restera aussi à intégrer correctement la gestion des réunions, avec inscription/désinscription, affichage des documents associés, et mise à jour automatique des documents via les triggers déjà créés. Ce module est encore à développer côté application.

Enfin, mes interfaces pourraient être améliorées pour gagner en ergonomie : un affichage plus clair des participants, une meilleure gestion des erreurs, et un tableau de bord plus synthétique pour les administrateurs.

À moyen terme, je pourrais aussi ajouter des fonctionnalités de contrôle supplémentaires, comme l'affichage ou la gestion des indisponibilités des ressources, ou encore des améliorations visuelles pour rendre l'expérience plus fluide.

En résumé, la base est solide et totalement fonctionnelle pour tout le périmètre du Sprint 1 et de la première partie du Sprint 2. Les prochaines améliorations concernent surtout les interactions avancées autour des réservations, les réunions, et le perfectionnement général de l'interface afin d'obtenir une version complète et finalisée de ResaWeb.

➤ **Procédure à suivre pour installer l'application Web sur un serveur**

1. Télécharger l'archive ZIP du projet OU cloner le dépôt GitLab :

git clone <https://gitlab-depinfo.univ-brest.fr/e22507733/resaweb.git>

2. Extraire le projet dans le répertoire web du serveur, par exemple :

- Apache/Linux → /var/www/html/
- Wamp/Xampp/Localhost → C:\xampp\htdocs\

3. Importation de la base de données :

- Ouvrir phpMyAdmin ou un terminal MySQL.
- Créer la base :

CREATE DATABASE e22507733_db1 DEFAULT CHARACTER SET utf8mb4;

- Importer le fichier SQL : e22507733_db1.sql

4. Configuration CodeIgniter

- Aller dans :

app/Config/App.php

- Modifier la base URL :

```
public $baseURL = 'https://adresse_du_serveur/';
```

- Aller dans :

app/Config/Database.php

- Modifier les accès MySQL :

```
public $default = [
    'hostname' => 'localhost',
    'username' => 'root',
    'password' => 'le mot de passe de la bd',
    'database' => 'e22507733_db1',
    'DBDriver' => 'MySQLi',
    'charset' => 'utf8mb4',
];
```

5. Configuration des permissions

- Donner les droits en écriture aux dossiers qui recevront les fichiers uploadés :
 - chmod -R 775 public/bootstrap2/assets/images
 - chmod -R 775 public/bootstrap2/assets/pdf

Si c'est Apach:

chown -R www-data:www-data public/bootstrap2/assets/

6. Lancer le site

Aller à : <http://localhost/nomprojet/public/index.php>

ou sur serveur universitaire :

<https://obiwan.univ-brest.fr/~identifiant/>

Liste des prérequis techniques

– Serveur Web

- Apache 2.x ou Nginx
- Activation du module mod_rewrite (Apache)

– PHP

- PHP **8.1+**
- Extensions obligatoires :
 - mysqli
 - intl
 - json
 - mbstring
 - xml
 - curl
 - openssl

– Base de données

- MariaDB / MySQL 10.x ou supérieur
- Import complet du fichier SQL

– Structure des dossiers CodeIgniter

Le serveur doit pointer vers le dossier **public/**.

– Droits d'accès

Les répertoires qui reçoivent des uploads doivent être accessibles en écriture.